

IKT215 Assignment 1: Gaussian Naïve Bayes Classifier

Student: Gomery K. Wanjiru
Course: IKT215

Disclaimer	on	Generative	AI
-------------------	-----------	-------------------	-----------

In accordance with UiA's guidelines, I declare that I used Generative AI (Google Gemini) to assist in organizing the structure of the code classes, generating the interactive Plotly visualization code, and formatting the mathematical equations within this report. The core logic of the Gaussian Naïve Bayes implementation and the specific analysis of results are my own work.

Introduction

This assignment focuses on implementing a Gaussian Naïve Bayes (GNB) classifier from scratch. The main goal was the probabilistic foundations of the algorithm, specifically utilizing Bayes' Theorem and the assumption of independence among features. I implemented the algorithm in Python, tested it on a synthetic binary dataset, and extended it for multi-class classification using the Iris dataset.

Methodology

I built a Python class `GaussianNBClassifier` that operates independently of `sklearn`'s pre-built models.

Mathematical Foundation

The classifier assigns a class y that maximizes the posterior probability $P(Y = y|X = x)$. Following the "naïve" independence assumption, this is proportional to:

$$P(Y = y) \prod_{i=1}^d P(x_i|Y = y) \quad (1)$$

where $P(Y = y)$ is the prior probability (calculated as the frequency of the class in the training data) and $P(x_i|Y = y)$ is the likelihood.

Implementation Details

- **Training (fit):** The model calculates the mean (μ) and variance (σ^2) for every feature for each specific class. This defines the Gaussian distribution for each attribute.
- **Prediction (predict):** I implemented the Gaussian Probability Density Function (PDF) to calculate feature likelihoods:

$$P(x_i|Y = y_j) = \frac{1}{\sigma_{i,j}\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x_i - \mu_{i,j}}{\sigma_{i,j}}\right)^2\right) \quad (2)$$

The total likelihood is the product of these individual feature probabilities. The model then selects the class with the highest product of the prior and total likelihood.

Key Findings and Results

Task 1: Synthetic Binary Dataset

Using 1000 samples and 2 features, the model performed effectively.

- **Accuracy:** Approximately 90-93% (depending on random state).
- **Visualization:** Interactive Plotly decision boundary plots showed a clean separation, though some overlap was visible where the Gaussian distributions intersect.

Task 3: Iris Dataset (Multi-class)

- **Scenario A (2 Sepal Features):** Accuracy was roughly 80%. The model struggled to separate the Versicolor and Virginica classes due to significant feature overlap in sepal dimensions.
- **Scenario B (All 4 Features):** Performance jumped to nearly 100%. Including petal dimensions provided the necessary discriminatory power for perfect classification.

Challenges and Solutions

Numerical Stability: Multiplying several small probabilities can lead to arithmetic underflow. While I used the direct product as specified in the assignment for these low-dimensional tasks, I recognized that for higher-dimensional data, summing log-probabilities ($\sum \log(P)$) is the standard solution.

Independence Assumption: Features in the Iris dataset (like sepal length and width) are naturally correlated. However, the GNB model remained highly robust, proving that the algorithm often performs well even when the "naïve" assumption is technically violated.

Conclusion

I successfully implemented the GNB classifier from scratch. The results demonstrate that while GNB relies on a simple independence assumption, it is a computationally efficient and highly effective algorithm for both binary and multi-class classification tasks.